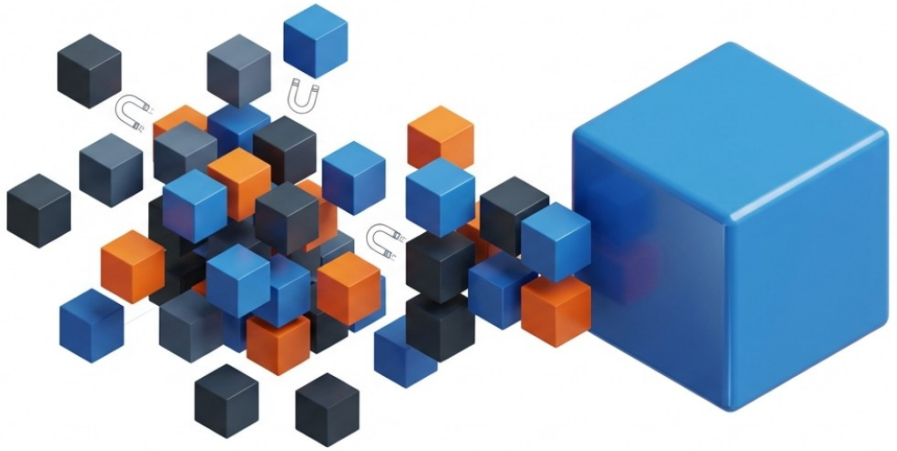
CHAPTER 6

SELECTION STRUCTURES IN C++

Chapter 3 • Selection Structures in C++

Ho Chi Minh City of University
Falculty: [Tên đơn vị]
Professor: [Họ tên]



LEARNING OBJECTIVES

After this chapter, students can…

1. **Explain program flow of control**

   Distinguish sequential execution from selection structures and describe when decisions are needed

2. **Use the bool type and logical expressions**

   Apply relational and logical operators, understand short-circuit evaluation, and form correct conditions

3. **Write if-else and nested if statements**

   Implement single-branch, two-way, and multi-way decisions while correctly handling block scope

4. **Apply the switch statement**

   Select among multiple cases using integral expressions, manage break and fall-through, and handle the default case

Content

1	Program Flow of Control
2	The bool Type and Logical Expressions
3	The if-else Statement
4	Nested if Statements
5	The switch Statement

Content

1	**Program Flow of Control**
2	The bool Type and Logical Expressions
3	The if-else Statement
4	Nested if Statements
5	The switch Statement

Program Flow of Control

What is Flow of Control?

- The order in which statements are executed at runtime
- Default: sequential execution (top to bottom)

When sequential is not enough

- Program often need to make decisions

→ This is called a selection structures

Other control structures in C++

- Repetition → loops
- Function invocation → call another function and return

Content

1	Program Flow of Control
2	**The bool Type and Logical Expressions**
3	The if-else Statement
4	Nested if Statements
5	The switch Statement

The bool Type

Boolean type in C++

```
bool variable;
```

Two possible values

- True
- False

Internal representation (ANSI/ISO)

- True → 1
- False → 0

Common use

- Store the result of comparison or logical expressions

Example: bool Type

```
#include <iostream>
using namespace std;
int main() {
     bool t1, t2;
     t1 = true;
     t2 = true;
     cout << “The value of t1 is” << t1 << “\n and the value of t2 is” << t2
<< endl;
     return 0;
}
```

Relational Operators

- Used to compare two values → result is bool

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Note:

- Non-zero value → true
- Zero → false

Logical Operator

Operator	Meaning
&&	AND
||	OR
!	NOT (negation)

A	B	A && B	A || B	! A	! B
true	true	true	true	false	false
true	false	false	false	false	true
false	true	false	true	true	false
false	false	false	false	true	true

Truth Table

Example: Logical Operators

```
int age = 45, term = 5;
int i = 3, j = 3;
int a = 2, b = 5;
bool complete = false;

// Results
(age > 40) && (term < 10);    // true
(age > 40) || (term < 10);    // true
!(age > 40);                  // false
(i == j) || (a < b) || complete; // true
```

Short-Circuit Evaluation

Definition:

- Evaluation stops as soon as the final result is known

Rules

- A || B → if A is true, B is not evaluated
- A && B → if A is false, B is not evaluated

Benefits

- Increase efficiency
- Avoids unnecessary computations
- Prevent runtime errors

Examples: Short-Circuit

OR example:

```
(a == b) || (c == d) || (e == f)
```

If `(a == b)` is true → the rest is skipped

AND example:

```
(a == b) && (c == d) && (e  ==  f)
```

If `(a == b)` if false → the rest is skipped

Operator Precedence

Level	Operator	Associativity
1	!, unary, -, ++, --	Right to left
2	*, /, &	Left to right
3	+, -	Left to right
4	<, <=, >, >=	Left to right
5	==, !=	Left to right
6	&&	Left to right
7	||	Left to right
8	=, +=, -+ …	Left to right

Content

1	Program Flow of Control
2	The bool Type and Logical Expressions
3	**The if-else Statement**
4	Nested if Statements
5	The switch Statement

The if-else Statement

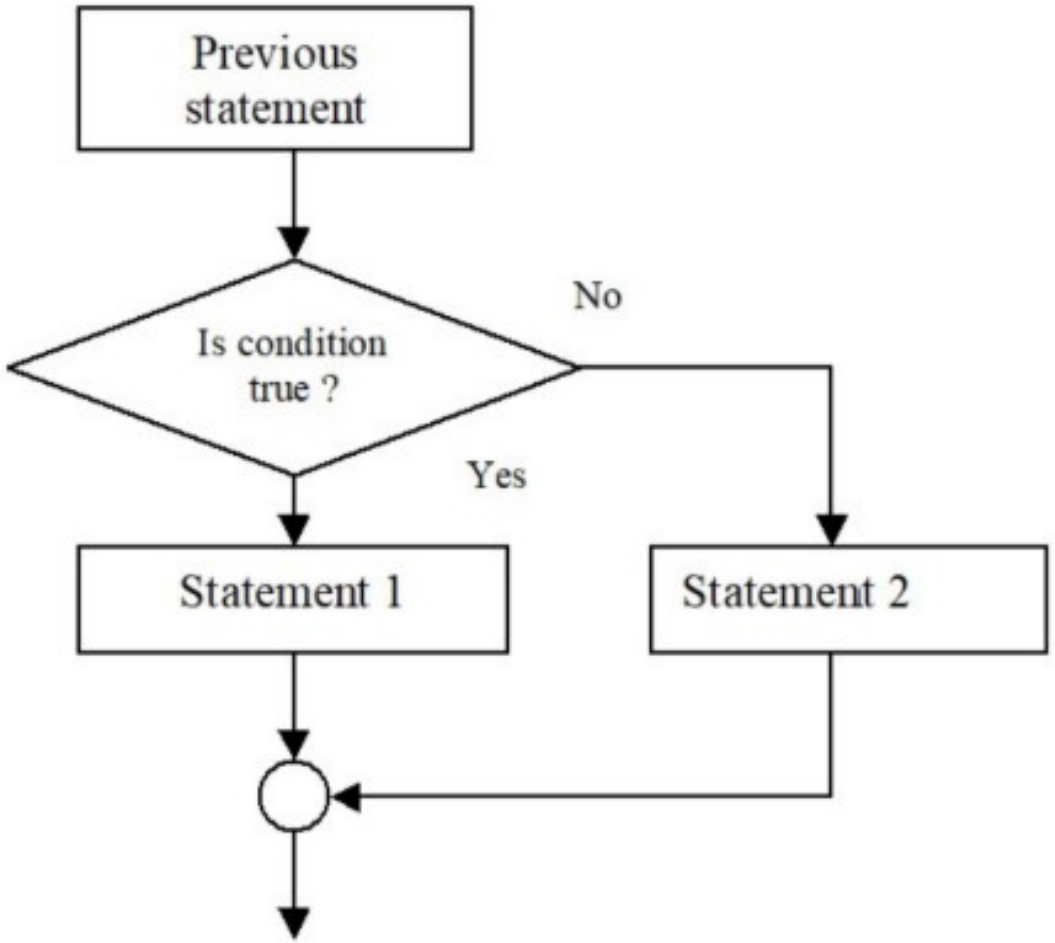
Purpose: Choose one of two execution paths based on a condition

Syntax:

```
if (condition) {
    // executed when condition is true
}
else {
    // executed when condition is false
}
```

Recommendation: Always use curly braces `{}` even for a single statement (better readability)

The if-else Statement



Flowchart

Variable Scope (Block Scope)

- A pair of {} creates a block
- Variables declared inside a block are visible only inside that block

Example – Shadowing

```
{
    int a = 25;
    {
        float a = 46.25;   // shadows outer a
    }
}
```

Benefits:

- Avoid name conflicts
- Reduces errors
- Makes code easier to understand

Single-Branch if

When there is no `else`?

```
if (condition) {
    // statements
}
```

Common use:

- Input validation
- Error detection
- Displaying warnings

Content

1	Program Flow of Control
2	The bool Type and Logical Expressions
3	The if-else Statement
4	**Nested if Statements**
5	The switch Statement

Nested if Statements

- Nested if = placing one if inside another
- Common form – if…else if…else chain

```
if (condition1) {
    // branch 1
}
else if (condition2) {
    // branch 2
}
else {
    // default branch
}
```

Content

1	Program Flow of Control
2	The bool Type and Logical Expressions
3	The if-else Statement
4	Nested if Statements
5	**The switch Statement**

The switch Statement

Purpose: Select and execute one off many code blocks based on the value of an expression

Syntax:

```
switch (expression) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
        // default statements
}
```

Important restriction: Expression must be of integral type (`char`, `int`, `short`, `long`…)

Cannot use `float`, `double`, or `string`

How switch Works

- Evaluate the expression (must be integral)
- Compare the value with each `case` label (in order)
- When a match is found → execute that case
- If no match → execute `default` (if present)
- If no `default` → skip the entire `switch`

The break Statement

Role of `break`

- Immediately exits the `switch`
- Control jumps to the statement after the `switch`

Without `break` → Fall-through

- The program continues into the next case(s)
- Sometimes useful, but usually an error

Example: Simple Calculator with switch

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2;
    char op;

    cout << "Enter two numbers: ";
    cin >> num1 >> num2;
    cout << "Enter an operator (+, -,
*, /): ";
    cin >> op;

    switch (op) {
        case '+':
            cout << "Result: " << num1
+ num2 << endl;
            break;
        case '-':
            cout << "Result: " << num1
- num2 << endl;
            break;
        case '*':
            cout << "Result: " << num1
* num2 << endl;
            break;
        case '/':
            if (num2 != 0)
                cout << "Result: "
                     <<
static_cast<double>(num1) / num2 <<
endl;
            else
                cout << "Error:
Division by zero!" << endl;
            break;
        default:
            cout << "Invalid operator!"
<< endl;
    }
    return 0;
}
```

Key Points Summary

- `bool` → `true` / `false`
- Relational operators → produce `bool`
- Logical operators → `&&`, `||`, `!`
- Short-circuit evaluation saves time and prevents errors
- `if-else` for two-way decisions
- Nested `if` / `else if` for multi-way decisions
- `switch` for multi-way decisions on integral values
- Always use `break` unless fall-through is intentional

Common Mistakes to Avoid

- Using `=` instead of `==`
- Forgetting `break` in `switch`
- Using non-integral types in `switch`
- Not checking division by zero
- Declaring variables with the same name in nested scopes without care

Thank you!